Homework Business

Group 24
Team Members:
Sione Daniels, Austin Haight, Noah Hamlet, Caleb Harmsen, Lilly Kounkel, Mason West, Han Zheng

Project Synopsis
An incremental management game where players complete homework to earn money and build an automated homework operation.

Architecture Description
The game is an incremental management game in which the player earns money by completing homework questions. The money can be used to purchase upgrades that increase the player's production rate. The player begins by manually answering simple homework questions to earn money. The money can then be used to buy upgrades that improves the player's ability to answer questions. Eventually the player should be able to answer the questions without doing anything via in-game 'AI'. The architecture is designed to keep the graphical user interface to be separated from the game rules and game state. This is for the reason to expand the application if needed without disrupting too much of the others.

There are four areas; the presentation layer, game logic layer, model layer, and the persistence layer. The presentation layer is responsible for displaying the information to the player and receiving player input. The game logic handles the rules and calculation of the games. The model layer stores the current state of the game; funds, reputation, production rate, and purchased upgrades. Lastly, the persistence layer is responsible for saving and loading the player's progress. This setup follows the general architecture described in the project's requirements.

Presentation Layer
The presentation layer provides the main game interface. The main screen displays the player funds, income per second, reputation, homework question, and available upgrades. The player can enter an answer to a homework question and submit it through the interface.

The presentation layer does not handle the inputs but rather sends that information to the game logic layer. This keeps this area focused on displaying information and collecting inputs.

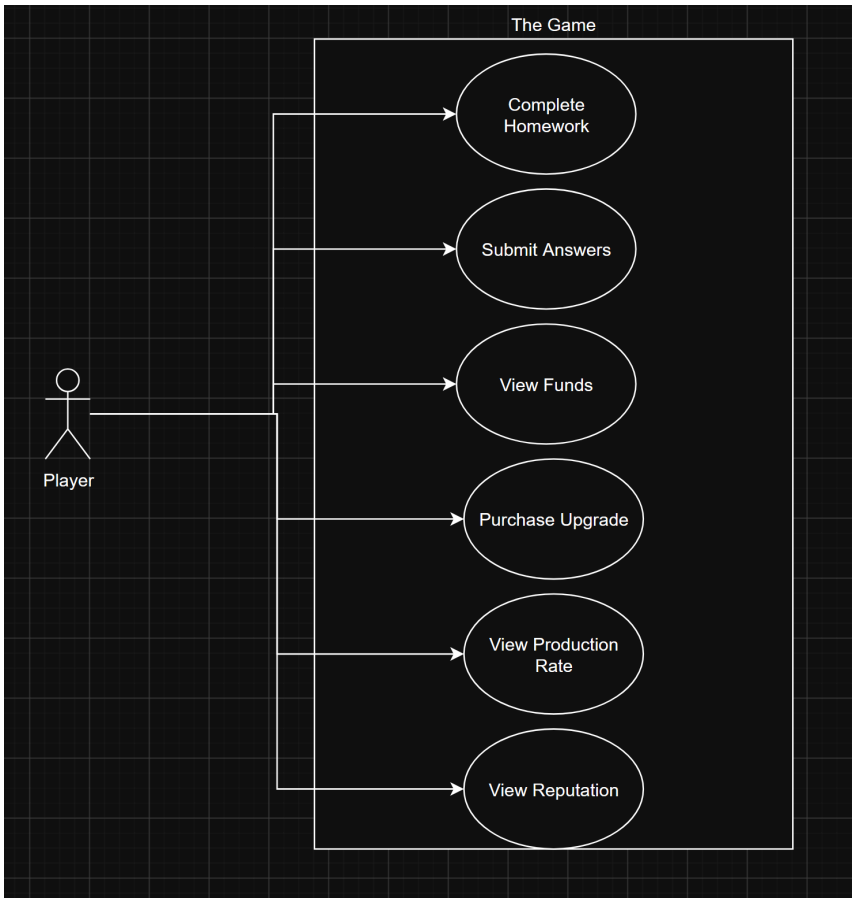


Figure 1: Use Case Diagram

Game Logic Layer
The game logic layer controls the rules of the game. When the player submits an answer, the game logic checks the answers. After processing, the game logic layer will take the necessary steps with the answers. For example, answering correctly rewards the player with money and reputation, or failing decreases reputation.

The game logic also controls the upgrades. Before an upgrade is bought, the system checks if the player has enough funds and if any required prerequisite has been met. As mentioned above, this layer is where the actual game takes place. This allows us to change the interface without worry.

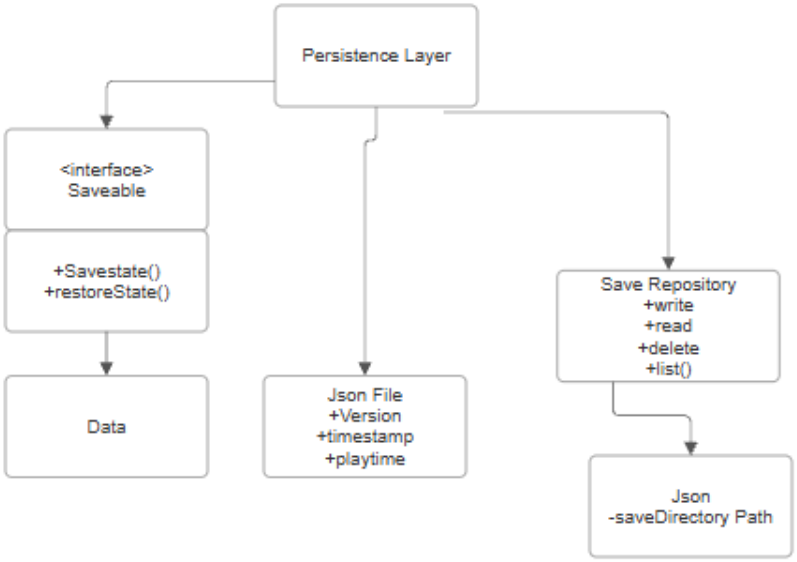
The game logic lastly controls automated production. Certain upgrades increase the player's income per second. When an upgrade is purchased, the production is updated immediately. This allows the player to see the effect without restarting the application.

Model Layer
The model layer stores the current state of the game. Information such as funds, reputation, income per second, value of completed homework, and the number of each upgrade owned. The model layer contains the information of the upgrades such as name, cost, owned, and production effect. Keeping this information in a separate game state allows the game logic to update the data without directly controlling how it is displayed. For example, if the player purchases a secondhand laptop, the model can update the number of laptops owned and the production rate. The presentation layer can then display the update values.

Persistence Layer
The persistence layer is responsible for storing the player's progress between game sessions. The project requirements identify saving funds, reputation, research, owned upgrade quantities, and the last played timestamp as later persistence requirements.

The initial implementation can use a simple local save file rather than requiring a database or cloud service. When the player saves or exits the application, the current game state can be written to a local file. When the application starts, the file can be loaded if it exists. If a save file is missing or corrupted, the application should start with a new game rather than crashing.

This is kept separated from the others so the storage method can be changed later if necessary.



Overall Architecture

The architecture flows between the four areas. The player interacts with the presentation layer. Player actions are passed into the game logic layer which checks the ‘rules’ then updates the model layer. The model layer informs the presentation layer and saves to the persistence layer if needed.

This architecture is intentionally simple because the first sprint focuses on creating a functional core gameplay loop. This consists of displaying the main game with the core gameplay features. The separation allows the team to work on the individual parts of the game without making the entire system dependent on one component. This would provide a foundation for the game for the remaining sprints.

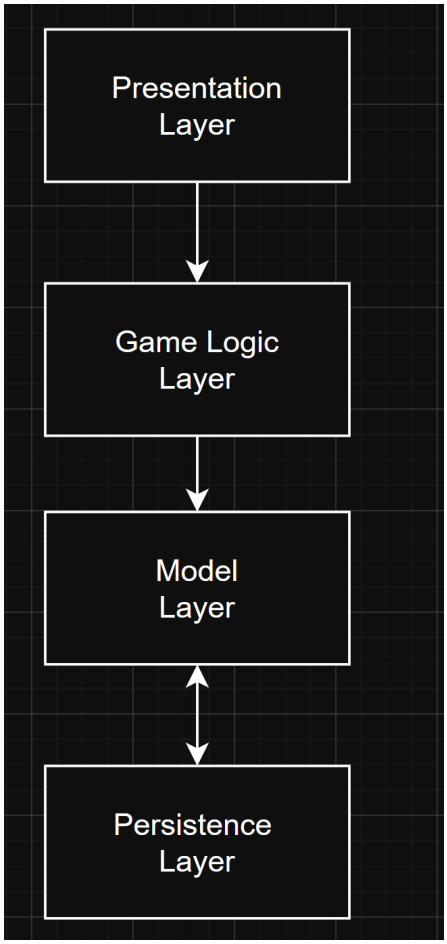


Figure 2: The layers between the 4